PROJECT REPORT

Draftly: Gmail AI Reply Agent

Full project report covering architecture, implementation approach, high-level design (HLD), low-level design (LLD), security, workflows, testing, and future enhancements.

Project Summary

Draftly is a backend-first email assistant that connects to Gmail, reads thread context, generates reply drafts using Gemini, stores draft history in SQLite, and only sends emails after explicit approval.

Technology Stack

FastAPI, Python, SQLAlchemy, SQLite, Gmail API, Gemini API, OAuth2, Tenacity, Fernet.

1. Introduction

Draftly was designed as a practical Gmail AI reply backend that solves a real workflow problem: repetitive email replies consume time, yet fully autonomous sending is risky without context, style alignment, and user control. The project focuses on building a reliable service layer that combines Gmail integration, prompt-driven reply generation, persistent draft lifecycle management, and send reliability features.

2. Problem Statement

Routine confirmations, follow-ups, acknowledgements, and scheduling responses are repetitive but still require context-awareness, correct threading, and appropriate tone. A useful AI assistant must therefore do more than generate text; it must understand thread context, preserve metadata, support review-before-send, and record operational events for traceability.

3. Objectives

- Integrate securely with Gmail using OAuth2.
- Fetch inbox messages and thread history via REST APIs.
- Generate AI-assisted reply drafts using Gemini.
- Use user preferences and recent sent emails for personalization.

- Persist drafts, send attempts, and logs in a database.
- Support approve, edit, reject, and send workflows.
- Ensure send reliability with retries and idempotency.

4. Scope

In Scope

- FastAPI backend
- Gmail OAuth and mailbox APIs
- Gemini-based draft generation
- SQLite persistence
- Review and send workflow
- Logging and status tracking

Out of Scope

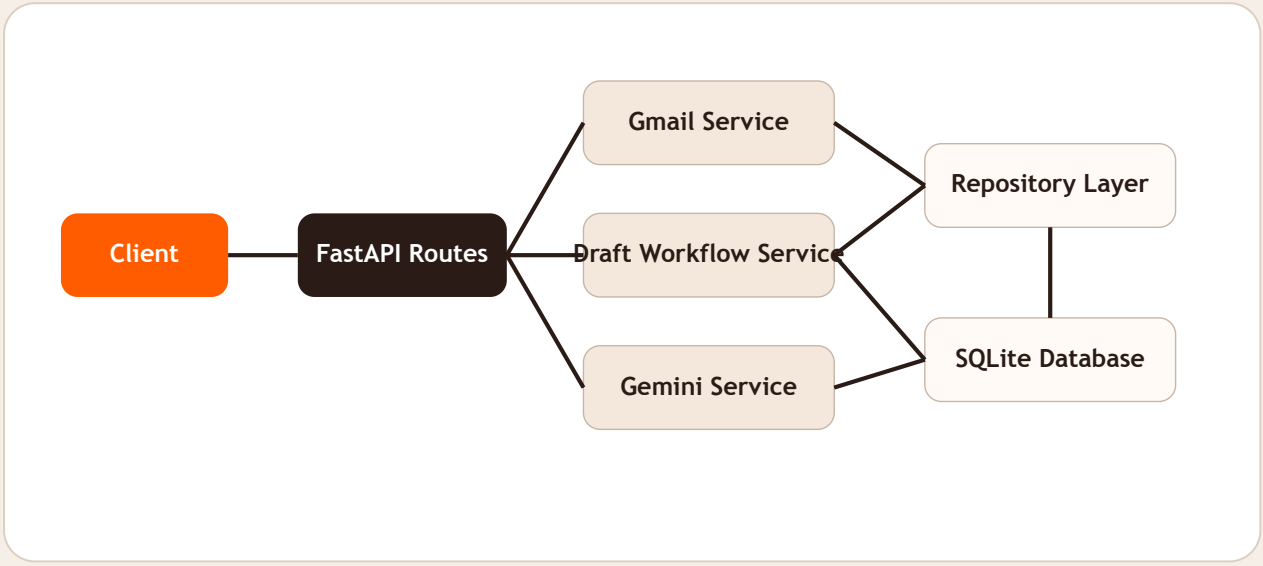
- Production frontend
- Cloud deployment automation
- Background worker infrastructure
- Advanced observability dashboards
- Team collaboration features

5. Functional and Non-Functional Requirements

Category	Requirements
Functional	OAuth login, inbox fetch, thread fetch, preferences save, draft generation, review, send, log access.
Security	Encrypted credentials and preferences, OAuth token revocation, explicit approval before send.
Reliability	Idempotent draft/send requests, retry mechanism for transient send failures, stored status transitions.
Maintainability	Separation of concerns across routes, services, repositories, and models.

6. High-Level Design (HLD)

The system follows a layered backend architecture. Clients interact with FastAPI routes. Those routes delegate work to service classes. The Gmail service owns mailbox and OAuth responsibilities, the Gemini service owns prompt generation and LLM calls, and the draft workflow service coordinates cross-service logic. Persistence is handled through repositories and SQLAlchemy models on top of SQLite.



> **HLD summary:** FastAPI acts as the orchestration layer, Gmail and Gemini are external system integrations, and SQLite provides durable storage for account state, drafts, send attempts, and audit logs.

7. Main Modules

Module	Responsibility
`app/main.py`	Route definitions, dependency injection, request validation, response mapping.
`app/services/gmail_service.py`	OAuth flow, Gmail client creation, message/thread retrieval, draft creation, sending.
`app/services/gemini_service.py`	Prompt preparation and Gemini reply generation.
`app/services/draft_service.py`	End-to-end orchestration of draft generation and sending workflows.
`app/services/repositories.py`	Persistence helpers, status updates, audit logging.

Module	Responsibility
`app/models.py`	ORM entities for users, OAuth state, drafts, send attempts, and audit logs.

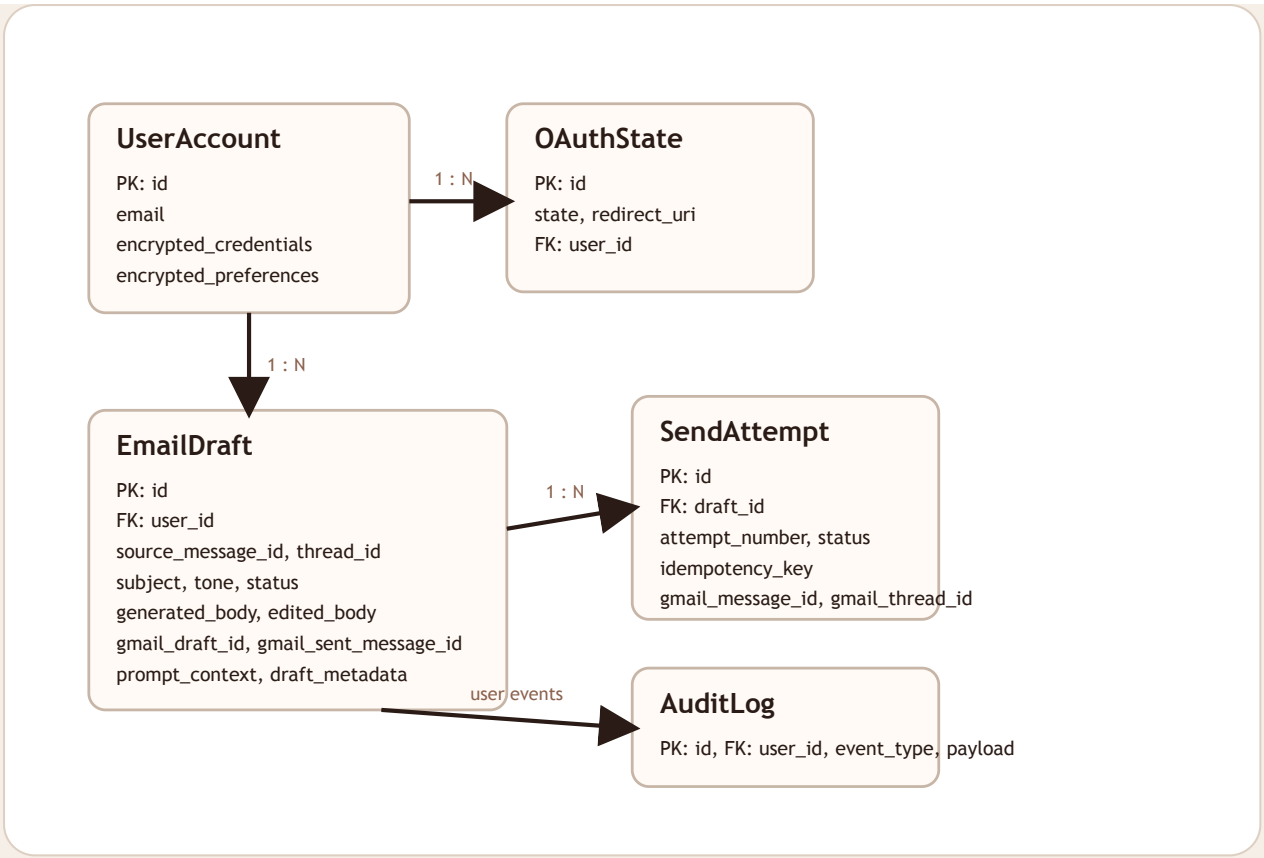
8. Low-Level Design (LLD)

The LLD describes how internal modules collaborate at the request, service, and persistence level. The backend keeps routing thin and moves operational logic into services and repositories.

8.1 Route-Level Design

Endpoint	Purpose	Core Module(s)
`GET /api/auth/google/start`	Create OAuth state and consent URL	main.py, gmail_service.py, repositories.py
`GET /api/auth/google/callback`	Exchange auth code and persist credentials	main.py, gmail_service.py, repositories.py
`GET /api/emails`	Fetch inbox message summaries	main.py, gmail_service.py
`POST /api/drafts/generate`	Generate and persist an AI draft	main.py, draft_service.py, gmail_service.py, gemini_service.py
`POST /api/drafts/{id}/review`	Approve, edit, or reject a draft	main.py, repositories.py
`POST /api/drafts/{id}/send`	Send an approved draft with retries	main.py, draft_service.py, gmail_service.py, repositories.py

8.2 Entity Design



The data model is centered around `UserAccount` and `EmailDraft`. OAuth states are temporary records used during login, send attempts preserve idempotent send history, and audit logs provide traceability across authentication, draft generation, review, and sending.

8.3 Draft Generation Sequence

1. API receives `POST /api/drafts/generate`.
2. User account is loaded by email.
3. Gmail service fetches the source message and its thread.
4. Gmail service fetches recent sent emails for style samples.
5. Gemini service generates a reply body from the compiled prompt.
6. Repository layer stores the draft and metadata.
7. Gmail service creates a Gmail-side draft for continuity.
8. Audit log is recorded.

8.4 Send Sequence

1. User approves or edits a draft.
2. API receives `POST /api/drafts/{id}/send`.
3. Draft workflow validates status and creates/reuses a send-attempt record.
4. Gmail message is built with reply headers and thread ID.
5. Send is executed with retry logic.
6. Draft, send-attempt, and log states are updated.

9. Security Design

- OAuth2 is used for Gmail access rather than storing passwords.
- Stored credentials and preferences are encrypted with Fernet.
- Only approved or edited drafts can be sent.
- Logout revokes the Google token and deletes the user account record.
- Audit logs preserve operational visibility for sensitive actions.

10. Testing and Validation

The project was validated with both code-level and runtime checks:

- Utility tests via `pytest`.
- FastAPI health endpoint validation.
- Local server boot validation with Uvicorn.
- Gemini response generation smoke test.
- OAuth-start flow validation after Google client configuration.

11. Challenges and Trade-offs

- Thread correctness required preserving Gmail headers and thread IDs.
- Human control was prioritized over autonomous sending.
- Lightweight style learning was chosen instead of a heavy personalization engine.
- SQLite was selected for simplicity, knowing production deployments would likely use a stronger database.

12. Future Enhancements

- Frontend dashboard for inbox and draft review.
- Background workers for retry queues and maintenance tasks.
- Observability metrics and dashboards.
- Richer user-level prompt templates and response profiles.
- Production-grade deployment with stronger persistence and secret management.

13. Conclusion

Draftly demonstrates a full backend architecture for an AI-assisted Gmail reply system. The implemented design integrates Gmail, Gemini, draft review, encrypted persistence, and send reliability into one cohesive workflow. The design remains intentionally modular so that the project can evolve into a larger product with UI, background jobs, and production deployment.